VIBE CODING

生成式软件开发-9

一 规范驱动开发(SDD)初探

大纲

01 规范(Spec)的基本概念
Specification Overview

02 软件开发方法的演进
Evolution of Software Development Methods

03 增量迭代与规范
Incremental Iteration and Specification

04 规范驱动开发（SDD）核心概念
SDD Core Concepts

05 开源SDD工具：OpenSpec
OpenSpec - Open Source SDD Tool

06 Trae /spec 模式
"/spec" Mode Practical Guide

01

规范（Spec）的基本概念

Specification Overview

从想法到软件

小明想把本学期要学的科目交给 AI 管理，让 AI 帮他整理每日学习计划。

听起来很简单，但打开电脑时… 他发现问题远没有想象的那么简单。

一个看似简单的想法，背后隐藏着大量需要明确的细节。

四个关键问题浮现：

谁可以用？

只有小明自己？还是能分享给同学？需要登录验证吗？

数据怎么存？

计划要不要保留？历史记录能查吗？能导出吗？

怎么用？

AI 自动分配？还是手动拖拽调整？界面操作是怎样的？

特殊情况？

哪天来不及学怎么办？临时加内容呢？平时上课或放假如何处理？

→ 从"我有一个想法"到"做出能用的软件"，最关键的不是写代码，而是**把想法说清楚**。

从想法到软件

想法天然是模糊的，但软件的实现需要精确。



从想法转化为软件

模糊的想法

"我想让AI帮我管理学习计划"

不清晰、不完整

→

细化描述

明确功能、约束、标准、边界

逐步细化

→

可运行的软件

AI按描述的内容生成

精确实现

- 一次一次交互（对话描述）
- 做Plan细化（单次计划细化描述）
- 书写规范（持续维护更新的描述）

什么是规范

规范（Specification）：关于 "应该做什么" "应该是什么样" 的明确约定

场景	规范1	规范2
装修	三室两厅、简约风、木地板	先铺防潮垫，再铺12mm橡木…
点菜	宫保鸡丁、少辣、不要花生	鸡腿肉切2cm丁，170°C滑炒30秒
西装	藏青色、修身版型、袖口绣首字母	前片面料斜裁45度、袖笼收窄1.5cm
活动	迎新晚会、300人、2小时、有抽奖	18:00放暖场音乐、18:30念开场白…

规范是否明确、是否细致决定了最终的结果是否能完美贴合需求

但挑战是很难（有时是不可能）一次性获得非常明确、细致的规范

规范的细化

核心挑战

太粗 → 失控

"做一个学习计划系统"

AI不知道具体要做什么，会自行发挥，结果可能与预期相差甚远。



太细 → 繁琐

规定每个按钮的像素位置和颜色值

过度约束AI的发挥空间，花费大量时间在不必要的细节上。

经验法则

写到 "不同人（包括 AI）读完不产生歧义"

- 预感到有歧义 → **加一句约束**
- 采用约定的表述形式 → **例如用户故事、Given/When/Then**

规范四要素

要素	说明	示例（学习计划管理工具）
功能要求	软件能做什么	"学生可录入要学习的科目和章节"
约束条件	有什么限制	"每人只能查看和编辑自己的计划"
质量标准	怎样算合格	"AI 生成每日计划在 10 秒内返回"
边界情况	异常如何处理	"单日任务超平时 2 倍时自动提示调整"

一份完整的规范应该包含这四个要素：

- **功能要求**定义了"做什么"，**约束条件**定义了"限制是什么"
- **质量标准**定义了"怎样算好"，**边界情况**定义了"异常怎么办"

文档化的价值

如果不写文档，知识会“蒸发”

单人开发

两周后你自己也看不懂当初的设计，成为自己的"陌生人"

多人协作

沟通成本剧增，陷入"代码考古学"的困境

后期交接

接手者只能根据代码推测意图，理解成本高

使用 AI 开发

AI 擅长补全模式，**不擅长读心术**

*"AI models are excellent at pattern completion but **poor at mind reading**."*

D. B. Piskala, Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants, https://arxiv.org/html/2602.00180v1

02

软件开发方法的演进

规范角色的变迁：从瀑布到敏捷

瀑布模型：一次写全



核心： 每个阶段按照顺序的方式进行

规范角色： 一次性交付品，大量的文档

问题： 软件开发中，**需求变化是常态**

> 💡 **启示：** 提前想清楚是好的，但试图 "一次全想清楚" 往往不现实。
>
> 瀑布模型假设需求在开发前就可以被完全确定，但软件项目的实际情况是：市场在变、用户在变、技术在变。

瀑布模型为什么出问题

制造业、建筑业需求基本清楚，很少变化 → 瀑布合理

但软件不同:

📈📈 市场变了

用户需求随市场环境快速变化

💬💬 用户反馈变了

看到原型后才真正知道自己要什么

🏃🏃 竞品出新功能了

需要快速响应竞争压力

最终结果

等到软件交付时，几个月前写好的需求文档已过时。

瀑布模型的根本假设——"需求可以在开发前被完整确定"——在软件领域几乎从不成立。

这催生了更灵活的开发方法：**敏捷开发**。

敏捷开发：拥抱变化

"可工作的软件，胜于面面俱到的文档。" – – 敏捷宣言

不是文档不重要，而是文档够用就行（Just Enough）。

敏捷做法：

- **大项目** → **小迭代**（1-4周一个周期）
- **每次只详细规范当前一小部分**
- **用户故事卡片代替大部头**文档
- **文档只记录关键约定**，不追求面面俱到

敏捷迭代示意



每个迭代包含：

- 规范当前功能
- 开发实现
- 用户反馈
- 调整下一迭代

敏捷的代价

规范角色变了： 从“ 大部头文档” 变成“ 每次迭代的便签纸”

✅ **灵活性大幅提升**

快速响应变化，持续交付价值

❌ **长期来看 → 没有人能完整说清系统应该做什么**

知识散落在：

聊天记录

开发者的记忆

代码本身

敏捷解决了“ 如何快速交付”的问题，但留下了

“系统的完整规范在哪里”的问题。

AI 生成式开发 & Vibe Coding

Vibe Coding

"大概描述一下我要什么，让AI 写代码，改到差不多就行。"

问题突出：

- AI 会加你没要求的功能
- 技术选择你完全不了解
- 同一需求两次结果不同
- 没有任何文档记录意图

*"Raw AI generation without structure produces **chaos**."* — *GitHub spec-kit*

三个时代的规范对比

维度	瀑布模型	敏捷开发	AI 生成式开发+敏捷迭代
规范形态	详细的需求文档	用户故事卡片	结构化可执行规范
写完时间	开发前完成	每个迭代前按需更新	持续演进、按需细化（也有迭代）
维护方式	写完人工修改	通过验收测试保障	AI和人一起维护，与代码同步
详细程度	巨细无遗	够用就好	逐步完善细化准确
主要挑战	跟不上变化	知识散落	AI大量产出和人审核效率的矛盾

螺旋上升：从 **“重”** 到 **“轻”** 再到 **“完善”** **“准确”**

03

增量迭代与规范

做一个版本 → 获得反馈 → 改进 → 做下一个

为什么软件不能一次做完

回到小明的学习计划生成器– – 一口气列出所有功能：

- 录入科目、AI 生成每日计划 • 学习进度追踪、计划分享
- 智能复习推荐、错题本 • 学习数据统计、时钟计时...

关于变化：

🔗 需求会变

用户想要动态调整，不是固定分配

🧠 理解会变

开发中发现最初设计行不通

📈 市场会变

竞品已上线，你的"完整版"还没出来

人工开发的时代，快速开发一个可运行的软件获取用户的反馈是关键

为什么软件不能一次做完

回到小明的学习计划生成器– – 一口气列出所有功能：

- 录入科目、AI 生成每日计划 • 学习进度追踪、计划分享
- 智能复习推荐、错题本 • 学习数据统计、番茄钟计时...

关于精确：

具体逻辑细节复杂

需要非常详细描述才能做准确

内容繁多难以一次表述

功能要全部想清楚写出来非常困难

需求本身就有弹性

功能描述从根本上就很难精确

AI开发的时代，虽然可以快速开发，但是最终目标却不清楚，很可能做出的东西并不是需要的

基本思想原则仍然是增量迭代：做一个版本 → 获得反馈 → 改进 → 做下一个

最小可行产品（MVP）

最小可行产品（Minimum Viable Product，MVP）：

"虽然简陋但能用"，核心目的是验证想法是否靠谱

AI学习助手的例子：

版本	主要功能	目标	验证假设
V1.0	录入科目、AI每周计划	验证基本需求	"需要AI做计划"
V2.0	学习进度追踪、计划分享	提升用户粘性	"愿意持续使用"
V3.0	智能复习推荐、错题本	差异化竞争	"需要高级功能"

用最小的成本，最快的速度，验证最核心的假设。

不要追求一次做完美，而是尽快获得真实反馈。

⚠️ 每个版本周期内，目标和预期也会发生变化、细化

文档化：维持**不变**的需求描述，更新**变化**的需求描述，添加**新增**的需求描述

活文档（Living Documents）

传统文档的命运：

写完 → 开发 → 放一边 → 过时 → 废弃

这就是 文档腐化（Documentation Rot）

文档化是工程化开发软件的重要手段之一

- 文档需要更新，否则会成为谎言和误导
- 文档需要轻量，从而便于理解和更新
- 代码本身需要易读，自解释的代码本身就是很好的文档

AI生成式开发中也需要文档，甚至比人类开发所需更多

规范作为软件开发中的“活文档"

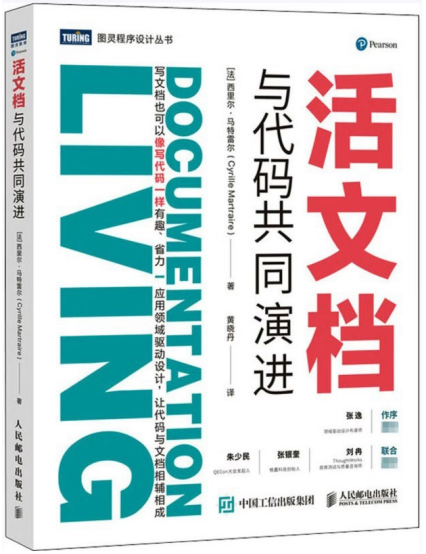
"SDD keeps the spec 'alive' and revisable, allowing updates that propagate through plan, tasks, and code."

– Microsoft Learn



活文档（Living Documents）

- 文档编写的是知识
- 文档是为了传递知识



活文档的核心原则

可靠

- 利用可用的知识
- 准确性同步机制

省力

- 简洁，一目了然
- 标准优于自定义
- 常青内容，维护少
- 自动重构更新

协作

- 交换知识，沉淀知识
- 易于获取（面向非技术人员）
- 集体所有权

有见地

- 谨慎的决定
- 嵌入式学习
- 真实性检查

SDD的Spec：也是一种活文档

- 在SDD中，规范不再是开发完成后的补充文档，而是驱动开发的核心工件。
- 基于规范去**实现**代码，而不是仅仅把规范作为代码的**注释**。
- 双向同步：当规范更新时，代码应该同步更新；当代码变更时，规范也应该反映这一变更。

AI学习助手：迭代1（MVP）

背景： AI 学习助手

用户故事

作为一个学生，我希望上传课堂笔记，由 AI 自动提取知识要点并生成思维导图。

验收标准

GIVEN 用户已登录并上传了课堂笔记
WHEN 点击"生成要点"
THEN 系统在 10 秒内返回 5-10 个知识要点
AND 系统生成一张可导出的思维导图

不做的事（排除项）

- ✗ 手写笔记识别（V2）
- ✗ 生成复习题（V3）
- ✗ 多人协作（暂不考虑）

用户反馈

"要点提取不错，但我更想要复习题！"

迭代2：需求深化

小丽发现：用户真正想要的是 **"考试准备助手"**，不是 "笔记整理工具"

🆕🆕 新增

用户故事： AI 根据我的笔记自动生成选择题和填空题

→ 从"整理工具"转向"考试助手"

✏️ 修改

产品定位： 从 "笔记整理工具" → "考试准备助手"

知识要点展示： 改为 "按考试重点排序"

🔒🔒 不变

笔记上传入口的位置和交互方式保持不变

🎯🎯 关键：对问题本身的理解，在迭代中深化了。

不是功能堆砌，而是对用户需求本质的重新认知。

迭代3

新需求出现 → **"错题追踪与智能推荐"**

迭代	核心功能	规范重点变化	驱动因素
V1	笔记→要点提取	确定基本框架	用户："更想要复习题"
V2	+ 自动出题	重新定义产品定位	用户："希望跟考试挂钩"
V3	+ 错题追踪& 推荐	加入个性化推荐	市场竞品的错题管理不好用

增量迭代

规范是一步步长出来的，每次迭代都在调整边界、修正理解、增加细节。

04

规范驱动开发（SDD）核心概念

"Specifications don't serve code — code serves specifications."

什么是 SDD

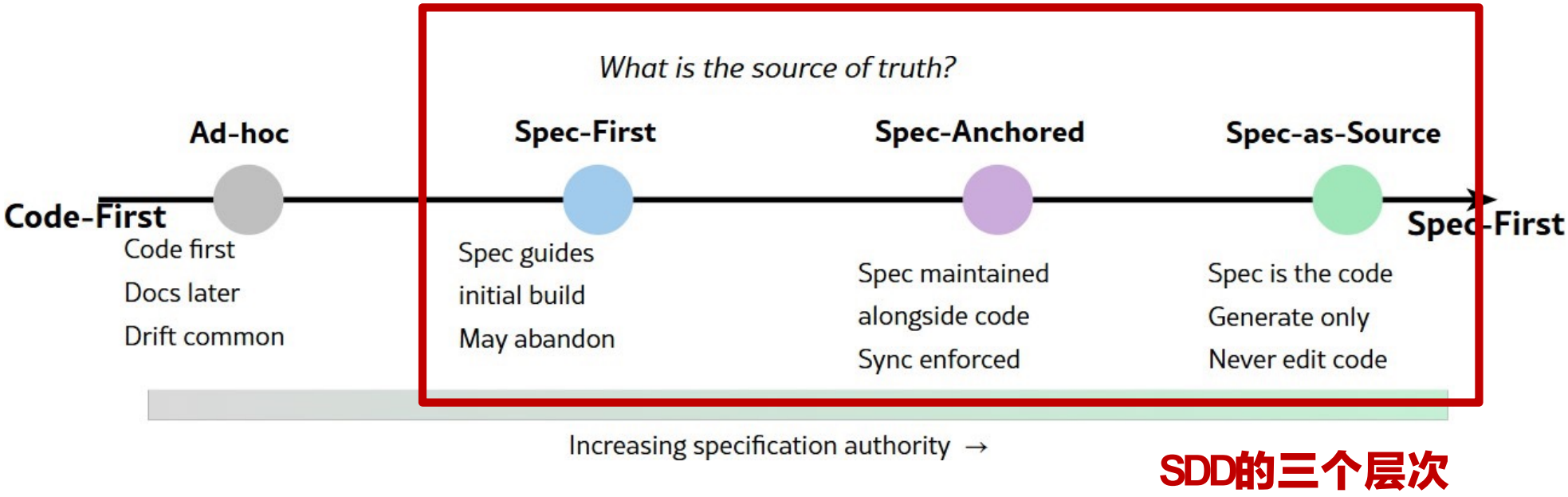
"Specifications don't serve code — code serves specifications." — GitHub spec-kit

权力反转:

方法	谁服务于谁	核心逻辑
瀑布	需求文档服务于代码	文档是开发的输入参考
敏捷	代码是真相，文档是辅助	可运行代码优先于文档
SDD	规范是真相，代码是规范的“ 翻译结果”	规范驱动一切

> **SDD =** 写代码之前，先和 AI 把 **"要做成什么样"** 讲清楚、写下来、达成共识。
>
> 规范是开发过程的**核心驱动力**，而不是事后补充的文档。

生成式开发方法的演进——从提示到规范



SDD的三个层次

D. B. Piskala, Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants, https://arxiv.org/html/2602.00180v1

SDD的三个层次

1. 规范优先

Spec-First / 引导初始开发

在交付给 AI 编写代码之前，**首先编写详细规范**（如 Markdown / PRD 格式），引导 AI 构建初始架构与核心逻辑。

特点： 往往在项目初建或特性初始开发阶段工作得很好。对原型构建、一次性特性开发非常有用。如果需要长期维护，那么因频繁的手动代码修改，极易发生“规范失效与漂移”（Spec Rot & Drift）。

2. 规范锚定

Spec-Anchored / 活文档与持续同步

将规范与代码**双向绑定**。通过 CI/CD 构建流水线与自动化静态检查，强制保证两者在软件整个生命周期内高度一致。

特点： 所有代码变更都必须在规范中先行体现。借助自动化工具链防止因盲目修改代码造成需求蒸发和架构失控。这是现在很多生产系统采用SDD方法的sweet spot。

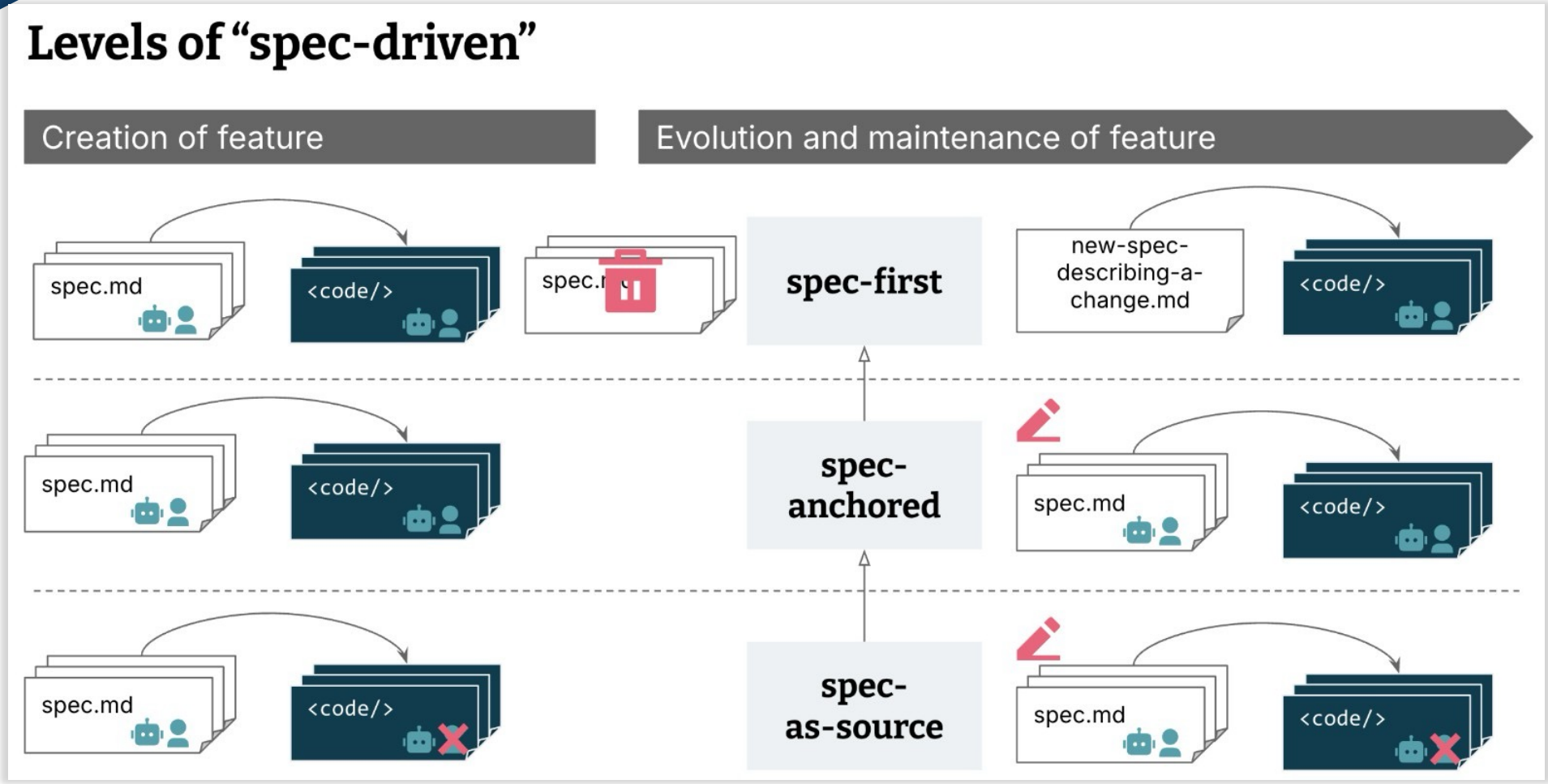
3. 规范即源码

Spec-as-Source / 人机分工

开发者**只修改规范本身**。真实生产代码完全由机器（AI）基于规范自动生成。开发者只考虑需求和行为，机器负责翻译为可执行的代码。

特点： 规范与代码由人和机器分别编辑，代码自动生成天然规避规范漂移风险。支持多语言或多架构的目标代码一次性转换生成。在代码生成能力可靠的特定领域，已是标准实践。但通用领域的代码生成往往缺乏这种可信度。

SDD的三个层次



Birgitta Böckeler, Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl, https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

Spec As Contract

📑 合同 = 双方对 "将交付什么" 的共同理解

🎯 条款清晰 → 不会产生歧义

✏️ 可通过补充协议修订，但必须有书面记录

✅ 验收依据 → **合同内容**，不是口头承诺

SDD 规范 = 人和 AI 之间的合约

- 回答的不是 "怎么做"，而是 **"做什么"**
- 双方（人和AI）对交付物有共同理解
- 可作为验收依据，逐条验证
- 可修订，但变更必须记录在案

规范的内容要求

01

可验证 ✅

每条要求都能被测试，确认是否 "做到了"

02

无歧义 🎯🎯

不同人（包括 AI）读完，理解应该一致

03

有边界 🚧

做什么 + **不做什么**（后者往往更重要）

有时还要记录为什么（决策过程）

04

可演进 📈📈

合约有明确的修订规则，持续更新演进，不是一成不变

SDD 的核心原则

以规范为通用语言
(Specifications as the Lingua Franca)

规范成为最核心的资产/产物。代码则演变为该规范在特定语言和框架下的具体体现。维护软件的本质，变成了持续演进其规范。

可执行规范
(Executable Specifications)

规范必须足够精确、完整且毫无歧义，能够直接据此生成可运行的系统。这消除了设计意图与最终实现之间的鸿沟。

持续精化
(Continuous Refinement)

一致性校验是一个持续进行的过程，而非一次性的关卡。AI 会作为一个常态化机制，不断分析规范中是否存在歧义、矛盾和遗漏。

研究驱动的上下文构建
(Research-Driven Context)

在制定规范的整个过程中，研究智能体（Research Agents）会持续收集关键的上下文信息，深入调研技术方案、性能影响以及组织层面的约束条件。

双向反馈
(Bidirectional Feedback)

生产环境的真实情况会反哺规范的演进。度量指标、故障事件和运维经验，都将成为优化和精炼规范的输入源。

通过分支进行探索
(Branching for Exploration)

基于同一份规范生成多种不同的实现方案，以此来探索不同的优化侧重点– – 例如性能、可维护性、用户体验或成本。

Github Spec-git, Specification-Driven Development (SDD), https://github.com/github/spec-kit/blob/main/spec-driven.md

SDD 的一般流程

创建规格说明

想法渐进具象化：从模糊、不完整的想法（Idea）起步，通过与AI的高效迭代对话，自动生成详尽的PRD。

自动化背景调研：AI研究代理（Research Agents）在此期间会针对库兼容性、性能基准、安全影响等深入调查并提供依据。

组织约束无缝嵌入：自动识别并集成公司数据库标准、身份认证要求及部署策略等约束条件。

制定实现计划

完成技术决策：基于PRD自动生成实施计划，将业务规则映射至架构细节，保证每项决定具备可追溯性。

规格与计划同步：当产品经理调整验收标准或架构师优化模式时，PRD与实施计划双向自动更新。

持续一致性校验：AI在流程中全天候分析规格说明，动态挖掘其模糊、矛盾和遗漏，非单次把关。

代码生成与测试

无需完全闭环：一旦规格和计划实现基本稳定，即可开始生成实现代码。早期生成具有探索性，尽快验证规范的可行性。

概念至代码转化：AI将领域概念转为数据模型，用户故事转API端点，验收场景直接转化为自动化测试。

测试/开发融为一体：测试用例不再是在编写完代码后补写，而是直接作为定义和生成实现的规格前置条件。

反馈循环与演进

运营闭环无缝反馈：生产环境中的指标及事故，会自动反馈并重塑为新的非功能性需求或架构约束。

规格说明更新再生：生产数据直接用于下一次规格的迭代更新与重新生成，而不仅用来发布紧急热修复。

颠覆传统SDLC流程：在“规格说明 - 实施代码 - 运营现实”之间构建持续进化循环，使传统SDLC全面升级。

Github Spec-git, Specification-Driven Development (SDD), https://github.com/github/spec-kit/blob/main/spec-driven.md

从创建规范开始

核心任务：模糊想法 → 清晰的功能规范

👥 用户故事

谁、想做什么、为什么

目标和背景，构建一个实际的上下文环境

✅ 验收标准

怎样算做完（Given-When-Then）

用测试把验收标准变得可执行，可判断真假

🚧 边界情况

特殊情况如何处理

无法一开始就完善，但人可以根据直觉与AI交互

❌ 排除项

本次不做的事（明确边界）

AI往往会规划出超乎预期的功能，并非都是必要的

⚠️ **AI通常可以帮助进行细化，但是一些细节还是需要人表达出来**

验收标准对比

❌ 差的验收标准

"预约功能应该好用"

→ 什么算 "好用"？太模糊！

→ 无法验证是否达标

→ 不同人理解不同

✅ 好的验收标准

GIVEN 用户已登录并选了A自习室

WHEN 选择 14:00-16:00 预约3号座位

THEN 生成"预约成功"确认

AND 该座位此时段对其他用户为"已占用"

AND 确认信息在 2 秒内返回

好验收标准的特征：

- 具体场景：描述了谁在什么情况下做什么
- 可观测结果：明确了系统应该如何响应
- 可自动验证：可以写成自动化测试
- 包含性能要求：响应时间、并发数等

SDD 与现有实践的关系

方法论	核心理念	与 SDD 的关系
测试驱动开发 TDD	先写测试，再写代码（红→绿→重构）	单元层面的 SDD。先写测试，本质上是在代码实现之前，编写一份定义预期行为的“微型规范”。SDD 将这种思维扩展到了更高的层面– – 功能、系统和架构– – 在更广泛的范围内应用相同的“规范先行”原则。
行为驱动开发 BDD	Given-When-Then 描述行为	现代 SDD 最直接的前身。Gherkin 场景是连接业务需求与技术实现的“可执行规范”。而 AI 辅助的 SDD 工具所带来的改变，在于能够协助从这些规范中直接生成代码，从而大幅缩短从场景描述到可用软件的路径。
领域驱动设计 DDD	领域概念分析与建模、上下文与架构映射	SDD 非常契合，因为它强调“通用语言 (Ubiquitous Language)”––即用开发人员和利益相关者都能理解的领域术语来编写规范。DDD 所倡导的这种共享词汇库，成为了构建对各方都具有实质意义的规范的基础。

D. B. Piskala, Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants, https://arxiv.org/html/2602.00180v1

SDD 的优势与劣势

✅ 优势

优势	说明
🎯 减少误解	规范 = 人-AI 合约
🔁 可复现	同一规范 + AI = 相似结果
📝 知识沉淀	长期、可检索的知识资产
🔍 可审计	逐条对照规范检查
🤝 协作友好	多人/AI 并行开发
🔧 拥抱变化	更新规范→重新生成

❌ 劣势

劣势	说明
⏱️ 前期投入	不适合 5 分钟原型
✍️ 写作门槛	需要练习 AI 能理解的写法
📏 粒度把控	太细束缚 AI，太粗 AI 乱猜
🔧 工具不成熟	生态快速发展，标准未统一
📚 维护成本	Spec-anchored 需持续更新

客观看待：

SDD 不是银弹，但它在 "AI 辅助开发" 场景下提供了结构化方法。

前期投入的时间会在后期维护、协作、复现中**成倍收回**。

SDD 的适用场景

✅ 适合 SDD

- 中等复杂度项目

（自习室预约、日常管理、小型 SaaS）

- 需要长期维护的项目
- 多人协作 / AI 辅助开发
- 有明确业务逻辑，规则清晰且较为复杂

（订单、权限、数据流程）

❌ 不适合 SDD

- 简单的快速原型验证

→ Vibe Coding 更快

- 一次性脚本 / 极小功能改动
- 纯探索性项目

（方向不确定，需要快速试错）

工具对比

工具	风格
OpenSpec	轻量、开放
Kiro	简洁、绑定IDE
spec-kit	重规矩、模板化
Tessl	激进、前瞻、实验性

OpenSpec轻量、易上手、与 TRAE 良好集成

05

开源SDD工具：OpenSpec

Fluid not rigid. Iterative not waterfall. Easy not complex.

OpenSpec 简介

OpenSpec 是一个为 AI 编码助手设计的**规范驱动开发工具**，它通过轻量级的工作流程，确保人类开发者和 AI 助手在编写代码之前就能**对需求、设计与任务达成明确共识**。

- ✓ 灵活迭代与低成本协作
- ✓ 减少需求漂移和上下文混乱问题
- ✓ 同时适用于从零开始的新项目与已有系统的增量开发与重构



Install OpenSpec globally:

```
npm install -g @fission-ai/openspec@latest
```

Then navigate to your project directory and initialize:

```
cd your-project
openspec init
```

OpenSpec 简介

Fission AI 开源项目，最受欢迎的 SDD 框架之一

设计哲学:

Fluid not rigid

开发过程灵活，可随时按需选择操作

Iterative not waterfall

增量迭代，每个变更一个小循环，而非一次性细化所有文档

Easy not complex

采用轻量 Markdown 文件，无需复杂配置

Built for brownfield

可在已有项目上工作（1→n），不必从零开始

OpenSpec 的理念:

- 用简单的 Markdown 文件管理规范
- 每个功能变更一个独立文件夹
- 支持从已有项目切入（brownfield）
- 与 AI 编程工具良好集成

Scalable from personal projects to enterprises

OpenSpec 希望既适用于个人开发者的小型项目，也能支持企业级团队协作。

OpenSpec 目录结构

目录树

```
openspec/
├──changes/
|  └──add-dark-mode/ ← 每个功能一个文件夹
|  ├──proposal.md ← 提案：为什么做、做什么
|  ├──specs/ ← 规范：需求和场景
|  |  └──theme-spec.md
|  ├──design.md ← 设计：技术方案
|  └──tasks.md ← 任务：可执行清单
└──archive/ ← 已完成归档
└──2025-01-15-add-login/
├──proposal.md
├──specs/
├──design.md
└──tasks.md
```

关键结构设计

changes/

- 存放进行中的功能变更，每个功能一个独立文件夹，其中有这个

archive/

- 已完成的功能归档，按日期命名

proposal.md

- 记录为什么做这个功能、要解决什么问题– – 用于说明变更的目标、范围以及整体思路。

specs/

- 详细的需求规范和场景描述

design.md

- 技术设计方案

tasks.md

- 可执行的任务清单

OpenSpec 工作流程

```
$ /opsx:propose add-dark-mode
```

AI 自动创建
openspec/changes/
add-dark-mode/

→

人审核修改
检查 proposal
确认 specs

→

/opsx:apply
AI 按 tasks.md
逐条实现

→

/opsx:archive
自动归档

AI 自动创建的文件：

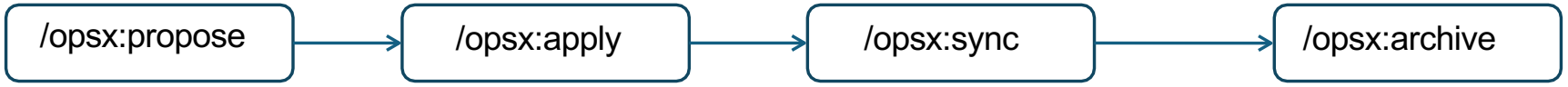
```
├──proposal.md ← "为什么要加暗黑模式"
├──specs/
│  └──theme-spec.md ← "各组件应呈现的颜色"
├──design.md ← "CSS 变量 + ThemeContext"
└──tasks.md ← [1.1 ThemeContext， 1.2 切换组件， 2.1 CSS变量， 2.2 localStorage]
```

Agree before you build → Build → Archive

OpenSpec 工作流程

✓ Default:

注：一般不需手动执行sync



创建一个新的变更提案　根据 spec 和 tasks 开始真正实现代码。　同步和更新 spec　归档已经完成的变更

✓ Expanded:



初始化一个新的 change　验证实现是否符合 spec。　归档已经完成的变更

继续之前未完成的 change

OpenSpec 主要指令

/opsx-propose

/openspec-propose

用于创建一次完整的需求变更，它会自动生成该变更的工作目录 ，并**提前产出实现前所需的核心规划文件**。

- ✓ proposal
- ✓ specs
- ✓ design
- ✓ tasks

直到变更进入“ 可执行状态”， 等待 /opsx:apply 开始开发。

/opsx-apply

/openspec-apply-change

用于执行一次已规划好的需求实现，它会读取 tasks.md 中的任务列表，识别未完成的任务，然后按照任务顺序**逐步推进实现过程**，包括**编写代码、创建或修改文件、必要时运行测试**，并在完成后将对应任务标记为已完成。整个过程是“ 任务驱动开发”，确保实现严格对齐 Spec 与设计。如果执行过程中被中断，也可以**从 tasks.md 的进度继续恢复**。

OpenSpec 主要指令

/opsx-archive

/openspec-archive-change

用于在一次需求变更完成后**进行归档处理**，如果发现任务未完成会进行提醒。同时它也会检查 Delta Spec 是否已经同步到主 Spec，如果没有同步，会建议用户先执行 sync 或在归档过程中一并完成。

在确认无误后，系统会**将整个 change 文件夹存档，保留所有设计与实现产物。**

/opsx-explore

/openspec-explore

用于开启一次无结构限制的探索式对话，主要帮助在没有明确需求前先**理解问题本身**。

- ✓ 读取并分析代码库
- ✓ 回答问题
- ✓ 对比不同方案与实现路径
- ✓ 不会生成任何正式的需求产物。

当思路逐渐清晰后，可以直接**切换到 /opsx:propose（默认流程）或 /opsx:new（扩展流程）**进入正式需求定义阶段。

增量规范（Delta Specs）

DELTA SPECS模板

Delta for Auth

ADDED Requirements

Requirement: Two-Factor Authentication

The system MUST require a second factor during login.

Scenario: OTP required

- GIVEN a user with 2FA enabled
- WHEN the user submits valid credentials
- THEN an OTP challenge is presented

MODIFIED Requirements

Requirement: Session Timeout

The system SHALL expire sessions after 30 minutes of inactivity.
(Previously: 60 minutes)

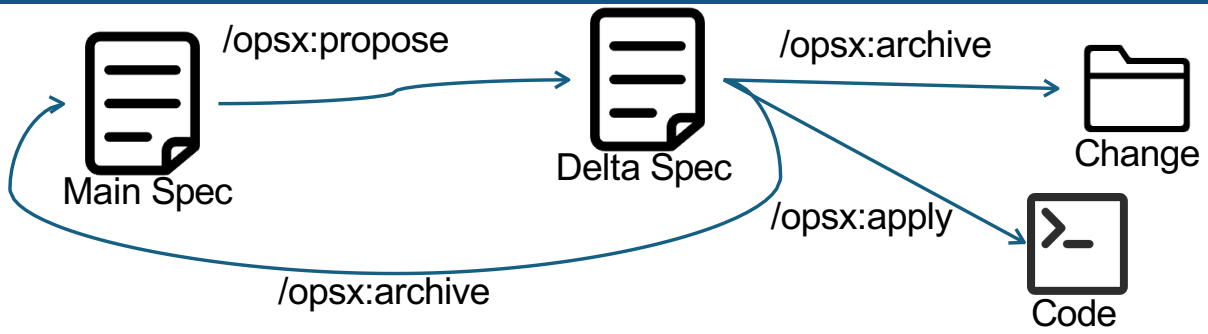
Scenario: Idle timeout

- GIVEN an authenticated session
- WHEN 30 minutes pass without activity
- THEN the session is invalidated

REMOVED Requirements

Requirement: Remember Me

(Deprecated in favor of 2FA)



DELTA SPECS中Requirements类型

REQUIREMENT类型	含义	示例
ADDED	新增需求	增加 2FA 登录
MODIFIED	修改需求	Session Timeout：60min → 30min
REMOVED	删除需求	移除 Remember Me

06

TRAE /spec 模式

TRAE/spec 模式是什么

TRAE 完成规范驱动开发的内置命令（command），提供类似于OpenSpec的能力

/spec 开发流程：

你输入需求
/spec ...

→

AI 生成
Spec + Tasks

→

你审核修改
确认规范

→

AI 按规范实现
编码 + 测试

简化了指令和OpenSpec流程
与Trae的集成度更高，交互更自然。

如何发起 /spec

示例命令

/spec 做一个自习室座位预约系统，学生可以：

- 注册登录
- 查看自习室实时座位状态与平面图
- 预约座位（选择座位和时段）
- 签到确认到场（超时自动释放）
- 查看自己的预约记录

AI 针对这个需求生成三个文件：

📋📋

Spec（规范）

需求描述、验收标准、边界情况、排除项

📝📝

Tasks（任务清单）

开发任务拆分、依赖关系、执行顺序

✅

Checklist（检查清单）

验证交付物是否合规的逐项检查表

评审 AI 生成的 Spec

AI 生成的规范只是初稿 — 需要你来审核!

评审维度	检查问题
🔍🔍 完整性	所有功能都在了吗？有无遗漏？
🎯🎯 准确性	AI 理解对了吗？有没有想复杂了？
🧪🧪 可测试性	验收标准能否用 "是/否" 判断？
🚧 边界覆盖	错误、空数据、并发冲突覆盖了吗？
❌ 排除项	AI 有没有 "好心" 加不需要的功能？
🏗️ 技术中性	Spec 不应有 "用 React 实现" 这类细节

修正示例

每条模糊表述 → 具体化为可验证的验收标准

❌ **如果规范是:**

"用户可以通过系统方便地预约座位"

→ 什么算“ 方便”？以什么为验收标准？无法确认任务是否完成了！



✅ **修正后**

用户可以选择自习室、查看座位平面图、点选空闲座位并选择时段后完成预约；

已被占用的座位以灰色显示且不可点选。

→ 具体步骤 + 可观测行为 + 明确的UI反馈

也可以要求AI以实例化需求的方式进一步细化验收标准和具体测试场景

人机协同修改流程

第1轮: 你 → /spec 想法

AI → 生成 spec / tasks / checklist 初稿

第2轮: 你 → "第3条验收标准太模糊，改成 Given-When-Then"

AI → 更新

第3轮: 你 → “整体功能比较多，分成两个阶段，第一阶段先实现……"

AI → 更新

第4轮: 你 → "checklist 漏了'手机端正常显示'"

AI → 补充

你确认 → 进入实现阶段

原则：你是决策者，AI 是协作者。改动由你发起、由你确认。

第1步：/spec 发起

背景： 学生在线查看自习室实时座位状态并提前预约

/spec 自习室座位预约系统

核心功能：

- 用户注册/登录（学号+密码）
- 查看自习室列表（总座位数、空闲数）
- 查看座位平面图（实时标注预约/使用中）
- 预约座位（选自习室、座位号、时段）
- 签到（扫码确认，超时未签到自动释放）
- 查看我的预约记录

不做：

- ✗ 在线支付（免费）
- ✗ 选多天时段
- ✗ 空调灯光控制（V2）

第2步：审核 Spec

用户故事1：注册与登录

✅ "用户使用学号+密码注册和登录"

✅ "注册时验证学号格式（8位数字）"

⚠️ AI 加了"支持手机号注册" → **划掉**（不在需求范围内）

⚠️ "密码不少于8位" → 加上"**必须包含字母和数字**"

补充：登录失败3次后锁定15分钟

用户故事2：预约座位

✅ "选择自习室、座位号、时段后完成预约"

⚠️ AI 的验收标准："用户成功预约后收到提示" → 太模糊！

→ 改为 Given-When-Then：

GIVEN 用户已登录并选了A自习室3号座位

WHEN 选择 14:00-16:00 时段并确认预约

THEN 显示"预约成功"并生成签到二维码

AND 该座位此时段对其他用户为"已占用"

补充：同一时段不能重复预约

第3步：审核 Tasks

AI 生成的任务清单，你审核后调整为：

1. **[P]** 搭建项目基础框架
2. **[P]** 实现用户注册登录模块
3. 实现自习室和座位数据模型（含实时状态）
4. 实现自习室列表和座位平面图展示
5. 实现座位预约功能（时段选择+冲突检测）
6. **[P]** 实现签到功能（超时自动释放）
7. 实现"我的预约"记录页面
8. 整体测试与修复

[P] = 可并行执行

你补充的依赖：

- 任务5依赖任务3

  （预约需要数据模型）

- 任务6依赖任务5

  （签到依赖已有预约）

AI 可能遗漏依赖关系，需要人工审核补充。

第4步：审核 Checklist

验收检查清单（你审核并补充后）：

- [ ] 注册：无效学号格式被正确拦截
- [ ] 登录：错误密码3次后锁定15分钟
- [ ] 自习室列表：空闲座位数与平面图一致
- [ ] 座位平面图：已预约座位实时标记为红色
- [ ] 预约：已被占用的座位提示"已被预约"
- [ ] 预约：同一时段重复预约被拒绝
- [ ] 签到：扫码成功，状态"已预约"→"使用中"
- [ ] 超时释放：15分钟未签到自动释放为"空闲"
- [ ] 我的预约：历史和当前分标签展示
- [ ] **所有页面在手机上正常显示（新增！）**
- [ ] **座位状态更新延迟不超过3秒（新增！）**

如果手工修改，那么务必让AI知道这个修改！

第5步

第5步：按 tasks 推进实现

- 每完成一个任务，对照 checklist 验证
- 发现问题 → **先改 spec，再改代码**（spec-first or spec-anchored）
- 所有任务完成 → 整体回归测试

SDD 完整闭环



🎯 **核心：让规范成为你和 AI 之间的共同语言**

不是各自脑补，而是基于同一份明确的规范协作。

另一个例子

AI学习计划生成器

AI学习计划生成器示例

用户输入：我想做一个AI学习计划生成器。 用户录入一些自己要学的内容，AI负责做整理，然后输出详细的学习计划，给到每一天。
请帮我做成一个可以运行的Web应用。

此时SDD的输出与我们的想法不同，故进行了一轮优化：

用户输入：我觉得这个学习计划应该和日历结合在一起，然后每天学习时间要定量，比如别超过6个小时，然后每一天点开可以和你的学习卡片差不多

这里我们可以看到SDD最大的用处：
帮助你理清思路，将你简单的话语转化为详细的，并且好理解的内容，来让我们把控AI的最终结果。

落到 Spec 后变成 5 类可实现承诺

01 学习内容录入
topic / notes / startDate / days / dailyHours

02 AI 计划生成
结构化逐日计划 + 6 小时上限

03 日历视图展示
月历、高亮、月份切换、tooltip

04 每日学习卡片
知识点、练习建议、总时长、关闭

05 导出与架构
文本导出、React + Express + localStorage

作用：把“生成计划”变成了前后端都能照着实现的合同。

SDD第一轮生成后

第一版“日历视图展示学习计划”中的SPEC没有明确是全局日历还是单计划日历，实现自然选择了更简单的单计划视角。

现象

生成 React 计划后看到独立日历；再生成机器学习计划后，又看到另一个独立日历。

用户无法在同一天看到多个计划的任务合计。

期望

用户是按天生活的，日历应该是一份全局视图。

5 月 20 日可以同时显示 React 2h + ML 1.5h = 3.5h。

给Trae回复

不同的任务使用的不同的日历，这和我们想要的并不相同，因为我们想要的是，一份日历，里面体现多任务，每日计划，也不应该只有一个任务要做。

Spec 的模糊性会直接导致后续实现的项目中出现问题。

SDD第一轮的修复

修复没有直接从代码开始，而是新建 unify-calendar-view spec，把“全局统一视图”写成新的可验收要求。



初始化生成器

单计划日历跑通，但视角偏差暴露。

记录偏差

明确“ 用户按天生活，不按计划生活”。

重写需求

日历展示所有已保存计划；同日任务合计。

实现修复

allDayPlans + DayTaskGroup + 自动保存。

- 在SDD下修复bug会给出一个新的spec，让人核对需要修复的内容
- 在人确定后，才会正式开始修复

让每一步的开发都有计划、细化的设计和任务，人都可以参与，可控性会更高；不过也同时也增加了人的审核工作量。

SDD第二轮迭代开发

在第一版AI学习计划生成器的基础上，我想增加能力。

1. 资料探索阶段
- AI在生成计划前，需要根据学生输入的学习内容生成搜索关键词；
- 系统需要为AI提供工具，例如Web搜索、网页内容读取、资料摘要；
- AI需要基于搜索结果形成对学习主题的初步理解；
- 系统需要记录使用过的搜索关键词、资料标题、URL和摘要。

2. 计划生成阶段
- AI基于整理后的学习主题、资料摘要和用户约束生成每日学习计划；
- 计划需要覆盖每一天，而不是只给出笼统建议；
- 每一天都应包含学习目标、时间安排、学习材料、练习任务和完成标准；
- 推荐材料必须尽量给出标题、URL和推荐理由。

3. 每日细化阶段
- 当生成某一天的详细安排时，AI可以继续围绕当天主题做更细的Web搜索；
- 每一天的材料和练习要尽量具体，而不是只写“学习某某知识点”；
- 如果某一天没有找到足够可靠的资料，需要在计划中明确说明。

4. 输出与前端
- 当前阶段先不做复杂前端编辑器；
- AI输出的学习计划可以先采用Markdown；
- 前端需要支持Markdown渲染；
- Markdown结构需要稳定，方便阅读和后续扩展。

迭代开发过程，给出比较完善的下一步需求。生成部分spec如下：（中间的一些澄清省略了）

```
## Why
第一版 AI 学习计划生成器让 AI 凭空规划，生成的计划缺乏真实资料支撑，内容笼统、不够合理。通过在生成计划前增加资料探索阶段，让 AI 先搜索和阅读真实学习资源，再基于可靠的资料摘要生成计划，可以提升计划的合理性和可信度。详细的材料推荐和练习任务则通过按需细化功能提供，避免一次性生成所有天的详细内容导致耗时过长。

## What Changes
- 后端新增 Web 搜索与网页内容读取工具，供 AI 在生成计划时调用
- 后端 AI 调用从单轮 prompt 改为多步 Agent 循环（资料探索 → 计划生成）
- 后端新增资料探索阶段：AI 自动生成搜索关键词、调用搜索工具、阅读和摘要资料，让计划更合理
- 初始计划生成保持简洁：学习目标、知识点、时间安排、练习建议（与第一版结构一致，但内容更合理）
- 后端新增按需每日细化 API：用户点开某天后点击按钮，AI 对当天做细化搜索，补充详细学习材料（标题、URL、推荐理由）和具体练习任务
- 类型系统扩展：DayPlan 增加 materials（细化后的推荐材料）、completionCriteria（细化后的完成标准）、dailyMarkdown（细化后的详细 Markdown）、refined 标记；LearningPlan 增加 exploration（探索记录）
- 前端 DayCard 组件增加"细化当天计划"按钮和 Markdown 渲染能力
- 前端新增探索结果展示区域，显示搜索关键词、资料标题、URL 和摘要
- 前端新增 react-markdown 依赖用于 Markdown 渲染
- 后端新增搜索 API 依赖（Tavily）
```

SDD第二轮引入测试

使用了自研的test-after-dev skill进行与Trae的spec可以共存的测试设计&开发环节

```
---
name: "test-after-dev"
description: "Performs comprehensive unit and E2E testing based on spec
after development. Invoke when development is complete, user asks to
test, or before committing changes."
---

# Test After Dev
试试 ⌘ U 与 AI 聊天, ⌘ I 与 AI 一起编写代码。
开发完成后的全面测试技能。根据当前开发所依据的 spec，自动识别可测试的需求，编写并执行单元测试和端到端测试，将测试用例设计文档和测试报告归档到 spec 目录的 test/ 文件夹下，测试代码放在源码目录中。

## 触发条件

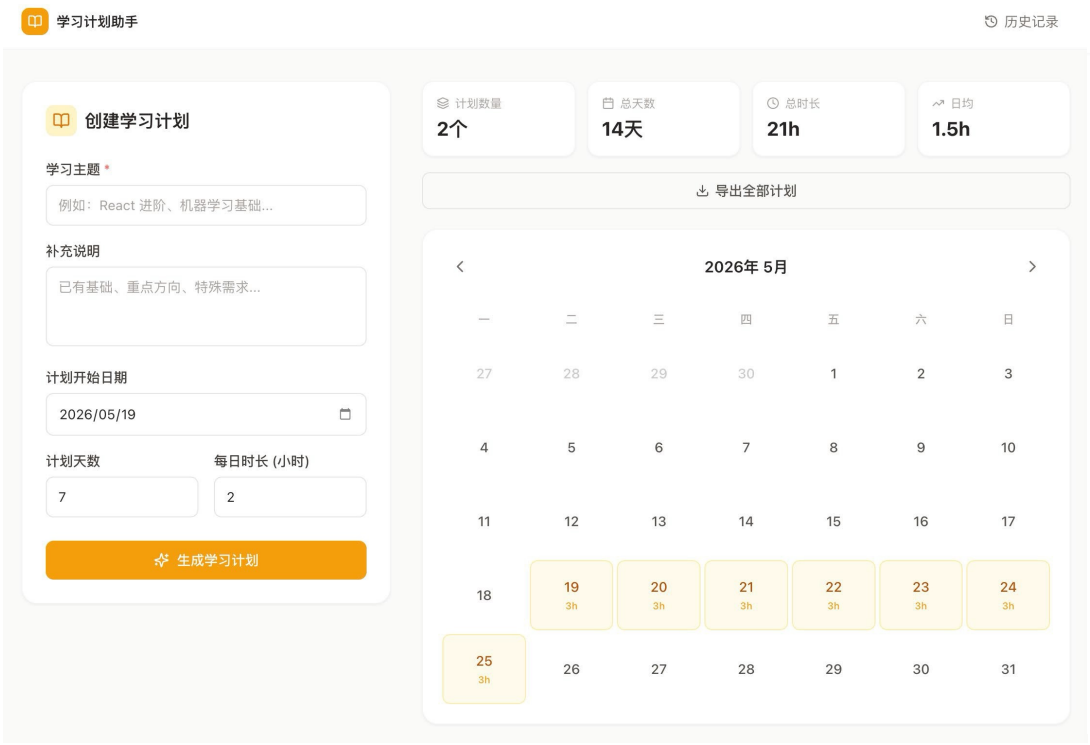
- 开发任务完成后，需要进行测试验证
- 用户明确要求进行测试（如"测试一下"、"写测试"、"跑测试"）
- 提交代码前需要验证功能正确性
- Spec 中有新增或修改的需求需要测试覆盖
```

Requirement: 资料探索阶段

Scenario	测试类型	测试代码位置	测试用例描述
资料探索成功	单元测试	api/services/search.test.ts	webSearch 正常返回搜索结果列表
资料探索成功	单元测试	api/services/reader.test.ts	readWebpage 正常提取网页正文
资料探索成功	单元测试	api/services/ai.test.ts	runExploration 调用搜索工具并记录关键词和资源
搜索工具不可用	单元测试	api/services/search.test.ts	TAVILY_API_KEY 未配置时 webSearch 返回空数组
搜索工具不可用	单元测试	api/services/ai.test.ts	探索失败时 exploration.searchFailed 为 true
搜索工具不可用	单元测试	api/services/reader.test.ts	readWebpage 请求失败时返回空字符串

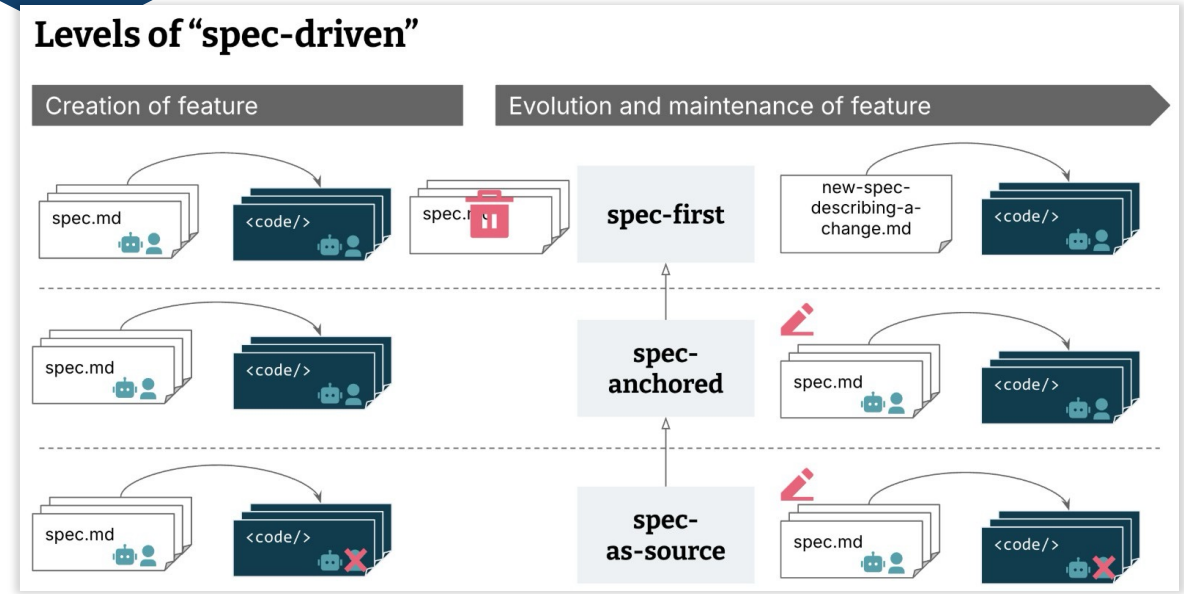
我们可以通过查看测试部分的测试用例，来把控AI做测试的完整度，比如说某一个场景下，如果AI没有进行覆盖，用户可以主动修改

最终效果


学习计划助手
历史记录
创建学习计划
学习主题 *
例如：React 进阶、机器学习基础...
补充说明
已有基础、重点方向、特殊需求...
计划开始日期
2026/05/19
计划天数
7
每日时长 (小时)
2
生成学习计划
计划数量
2个
总天数
14天
总时长
21h
日均
1.5h
导出全部计划
2026年 5月
一 二 三 四 五 六 日
27 28 29 30 1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 3h 20 3h 21 3h 22 3h 23 3h 24 3h
25 3h 26 27 28 29 30 31


学习计划助手
历史记录
5月20日 Wednesday
深度学习
看Coding Agent相关的论文
学习概览
深度学习
掌握反向传播和梯度下降的核心原理
损失函数与优化
0.5h
均方误差、交叉熵损失；梯度下降原理、学习率
反向传播算法
1h
链式法则、计算图、反向传播步骤推导
实战：简单神经网络前向与反向
0.5h
使用Python实现单个神经元的前向和反向传播（手动计算梯度）
练习建议
推导一个两层网络的梯度更新公式
实现一个小型神经网络的梯度下降训练
当日总学习时长: 2 小时

实践分享



*A practitioner's perspective on the iterative philosophy behind spec-anchored development — and knowing when to **switch between spec and vibe**.*

在实践中，“spec-anchored” 对我而言是这样运作的：

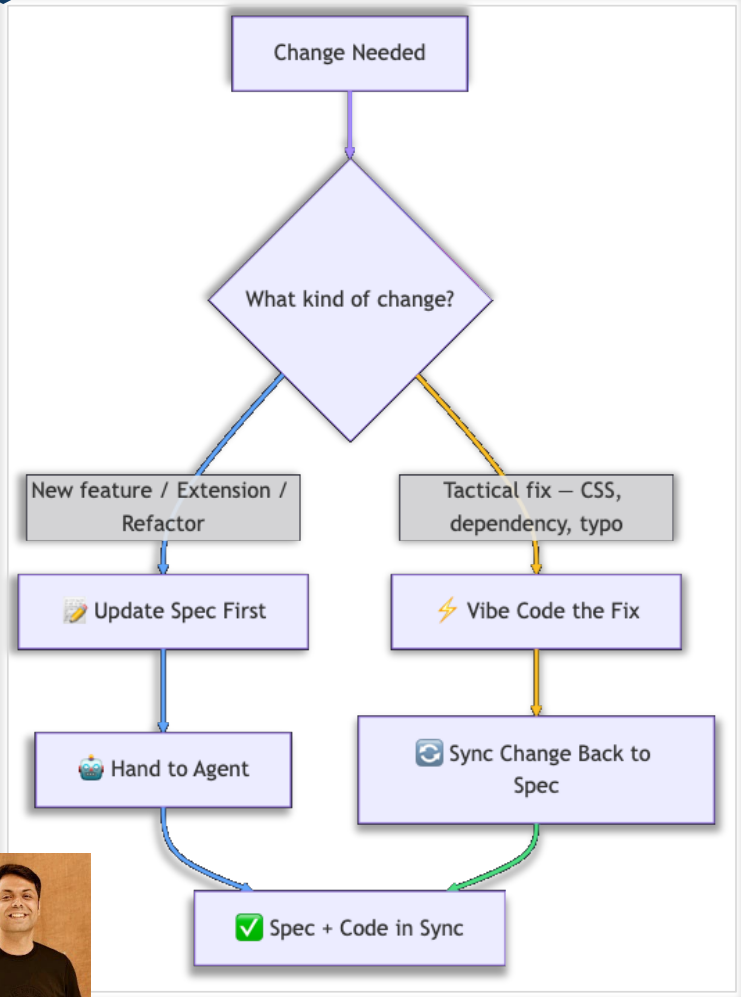
1. 我使用 Kiro 的规范工作流启动了一个 Python 项目——从需求、设计到任务，包含了全套流程。
2. 智能体（Agent）根据这些规范生成了初始代码库。
3. 在处理代码的过程中，我不可避免地进行了一些手动修改——比如开发过程中顺手做的修复、功能扩展和细节优化。
4. 最关键的一步是：**我将这些修改同步回了规范文档中**。每一次手动的代码改动，都会及时体现在需求、设计或任务列表中。

这种**反向同步（back-sync）**正是将“规范先行”转化为“以规范为锚点”的关键。规范不再是一次性的规划产物，而是变成了一份活的蓝图，随着项目的演进而始终保持准确。



Vivek Soni, Keeping the Spec Alive: A Practical Take on Spec-Driven Development, https://www.linkedin.com/pulse/keeping-spec-alive-practical-take-spec-driven-development-vivek-soni-ut46c/

实践分享





根据当前任务的复杂度和涉及范围，**在 spec-driven 和 vibe coding 之间灵活切换**是最有效的。它们并不是相互竞争的方法，而是相辅相成的技术。

– – 使用 spec-driven：

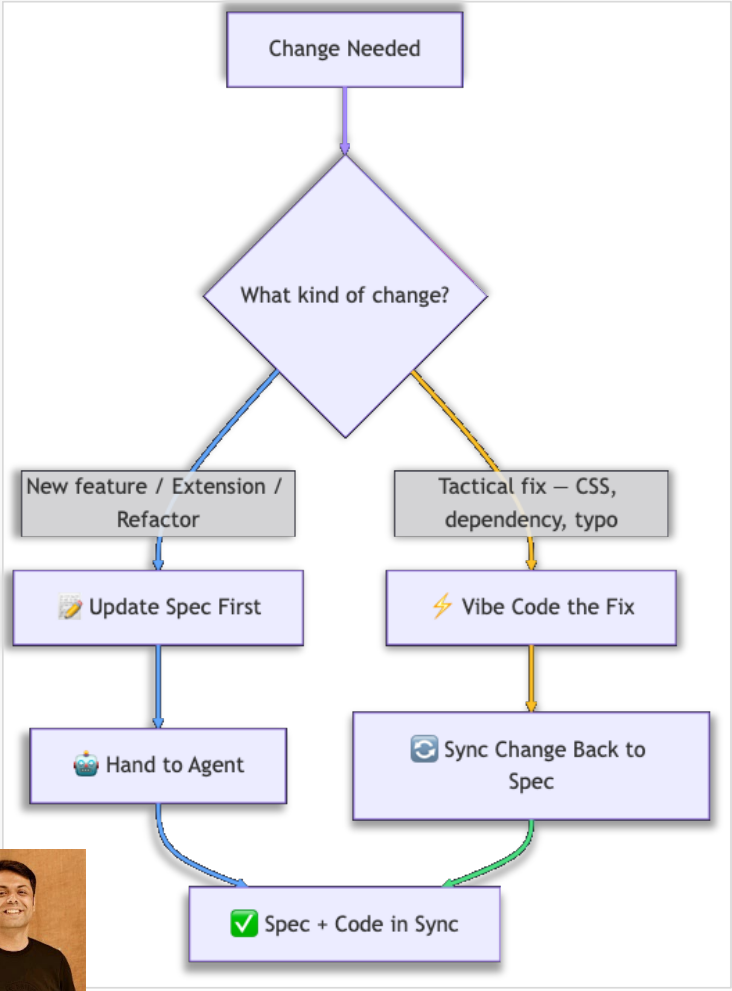
- 包含多个联动模块的新功能
- 影响整个系统的架构决策
- 需要具备可复现性或需要进行交接的工作
- 任何我希望智能体拥有深厚、结构化上下文的场景

– – 切换到 vibe coding：

- 战术性的微调或修复——比如 CSS 对齐问题、Python 依赖包的兼容性问题
- 小巧且自包含的改动，这类情况下写规范文档花的时间甚至比修复本身还要长
- 探索性的工作，此时我还不确定什么才是正确的方法

Vivek Soni, Keeping the Spec Alive: A Practical Take on Spec-Driven Development, https://www.linkedin.com/pulse/keeping-spec-alive-practical-take-spec-driven-development-vivek-soni-ut46c/

实践分享





在规范上的前期投入是完全值得的

- 保持规范准确所投入的时间，会通过减少返工、加快调试速度以及下游智能体更稳定的输出而加倍收回。

高于代码的动态信任源（source of truth）

- Spec-driven 开发并不是指永远不手写代码或绝不进行 vibe coding。它的核心在于拥有一个比代码更高层面的信任源– –一个能够捕捉设计意图而不仅仅是具体实现的载体。只有当你致力于在项目演进中保持这个信任源的生命力时，真正的价值才会显现出来。

反向同步（back-sync）**实现可复现性**（reproducibility）

- 一种简单的自律：每一步改动，无论是通过 spec-driven 还是 vibe-coded 完成的，都会被反向记录到规范中。正是这种习惯，才让以规范为锚点（spec-anchored）的开发模式在实践中真正行之有效。

Vivek Soni, Keeping the Spec Alive: A Practical Take on Spec-Driven Development, https://www.linkedin.com/pulse/keeping-spec-alive-practical-take-spec-driven-development-vivek-soni-ut46c/

课堂讨论

你开发软件时，分别什么时候用 /spec、/plan、vibe coding ？

到目前为止，你有什么常用的生成式开发的模式或者经验和大家分享吗？

你有什么自己很想做，但是做不出来的软件吗？你觉得主要原因是什么？
希望得到什么帮助？

End

9. 规范驱动开发(SDD)初探